ETH zürich

AST Lab

Automated Software Testing Project

Automated Reduction of Bug-Triggering Queries

Louis Meller / lmeller@student.ethz.ch

Kevin Collins / collinsk@student.ethz.ch

Contents

1 Introduction

Many programs are challenging to test thoroughly due to their large input spaces. A testing technique that is frequently used to combat this is fuzzing, specifically coverage-guided or structure-aware fuzzing. Due to its ability to automatically generate a large number of test cases to discover bugs, it can be quite effective, but one of the central issues with this methodology is that it can generate extremely large test cases that, although they trigger the bug, they often contain many irrelevant details that obscure the underlying cause. One way to tackle this problem is to employ test-case reduction techniques, automatically shrinking the test case while preserving the bug (i.e., the behaviour that made the test case interesting).

In this project, we evaluate the development of a test-case reducer, specifically designed to reduce SQL queries. Our system, when provided with an initial SQL query and an external oracle that checks if the query still triggers the interesting behaviour (usually a bug), attempts to minimise the query while satisfying the oracle. Our methodology takes inspiration from the delta debugging algorithm and uses a divide-and-conquer approach to generate candidate simplifications while continuously evaluating them using the oracle.

We evaluate our system in terms of both quality and speed across 20 sample queries. Then, we conclude by discussing the observed limitations and potential future optimisations of our system as well as the overall reduction strategy.

2 Design and Implementation

The core algorithm is implemented using alternating phases of the Delta Debugging (*ddmin*) algorithm [1] and a hierarchical node-collapsing algorithm. Our top-level reduction loop alternates between phases of statement-level *ddmin*, token-level *ddmin* and hierarchical node-collapse, until no further reductions can be made. The statement and token level *ddmin* steps aim to remove large parts of irrelevant tokens from the query in short time, keeping the work for the hierarchical node collapsing step small.

2.1 Delta Debugging Reduction

The *ddmin* algorithm uses a form of divide and conquer search to find a 1-minimal query that still satisfies the oracle. In our implementation, the algorithm operates by slicing a list of elements into N granular chunks per iteration, evaluating each chunk as well as its complement against the oracle. If a valid reduction is found, that is, if a smaller test case still satisfies the oracle, the whole process is repeated for this reduced query.

Otherwise, granularity N is doubled to test finer-grained reductions. To optimise execution speed on larger inputs, the algorithm terminates early once the chunk granularity satisfies $\text{len(query)}/N < \lceil \sqrt{\text{len(query)}}/5 \rceil$. This early-stopping criterion means that our *ddmin* phase intentionally avoids chasing costly single-element reduction steps, delaying this work for later reduction steps. Further this also means that our *ddmin* algorithm does not find a strictly 1-minimal solution.

2.2 Hierarchical AST-based Reduction

The AST-based algorithm, which was partly inspired by the Hierarchical Delta Debugging (HDD) approach [2], uses `sqlglot` to parse SQL queries into an AST, traverses the parsed tree, and then generates candidate replacements for individual AST nodes. Examples of candidate replacements include removing SQL clauses such as `WHERE`, `ORDER BY`, `GROUP BY`, `JOIN`, etc, removing parentheses, and simplifying boolean expressions. Then, each candidate query is converted back into SQL and reparsed to ensure that the mutation was valid SQL before it is validated with the oracle. The reducer greedily accepts the first candidate mutation that satisfies the oracle while simultaneously reducing the overall size of the query. There are also some intermediary steps we had to add in order to implement the described strategy effectively, such as stripping SQL statements and splitting them up based on semicolons. Overall, although the AST-based parts of the algorithm is more "SQL-aware" when compared to the *ddmin* part of the algorithm, resulting in the avoidance of invalid query generation. However it is bounded by a set of potential transformations, which may not be sufficient to reduce certain queries.

3 Results and Evaluation

The implementation of our reduction algorithm was tested across 20 benchmarks, SQL queries that triggered a specific bug. During evaluation, both the quality of reduction (in terms of reduction of tokens per query) and the speed of reduction (in terms of wall-clock execution time) were measured and analysed.

3.1 Quality of Reduction

The quality of reduction across the 20 benchmarks, measured as the reduction in tokens compared to the original query, can be seen in Figure 1. The average token reduction is at about 46.27%, but there is quite a lot of variance between queries. Several queries, such as *Q6*, *Q8*, and *Q20*, reduce extremely well, with the reducer achieving more than 93% token reduction on those queries. However, on the other hand, there are some queries were

reduction was minimal. Queries 3 and 14 were barely reduced (<2% token reduction), and the system failed to reduce *Q12* completely. After closer inspection of these queries, we see that the reducer does not perform well on very large test cases where the majority of the test case is built up on `INSERT` statements. This indicates that the reducer is sensitive to query structure, and more aggressive AST mutations might have performed better on such queries. Overall, the system works effectively on certain queries, but the reduction strategy needs to be adapted to cover a wider range of query structures.

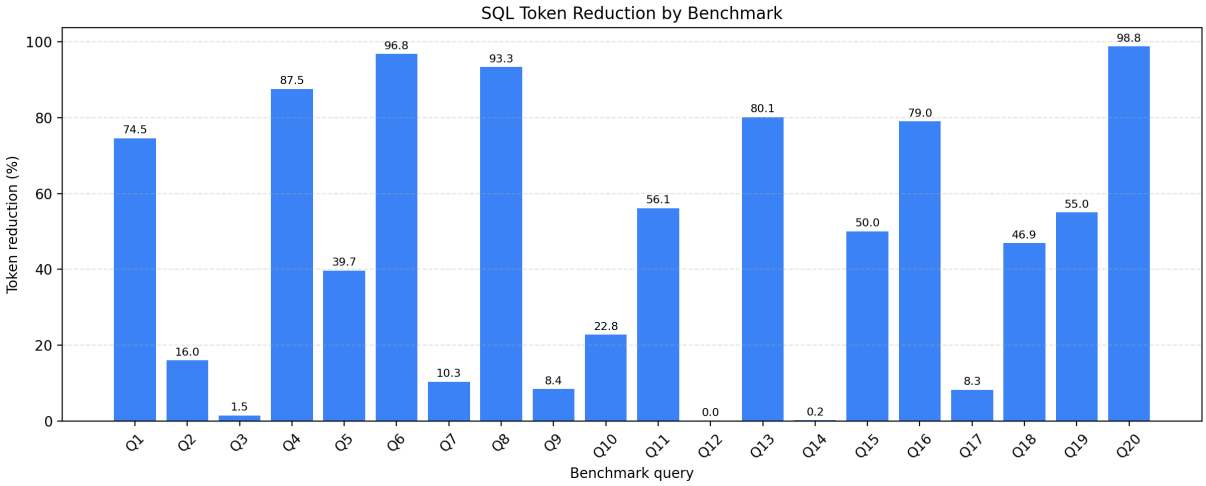

Figure 1: Quality of reduction (token reduction percentage) across 20 benchmarks.

Note: The data is also presented in a tabular form in Appendix A.

3.2 Speed of Reduction

The speed of reduction across the 20 benchmarks, measured as the wall-clock reduction time for each query, can be seen in Figure 2.

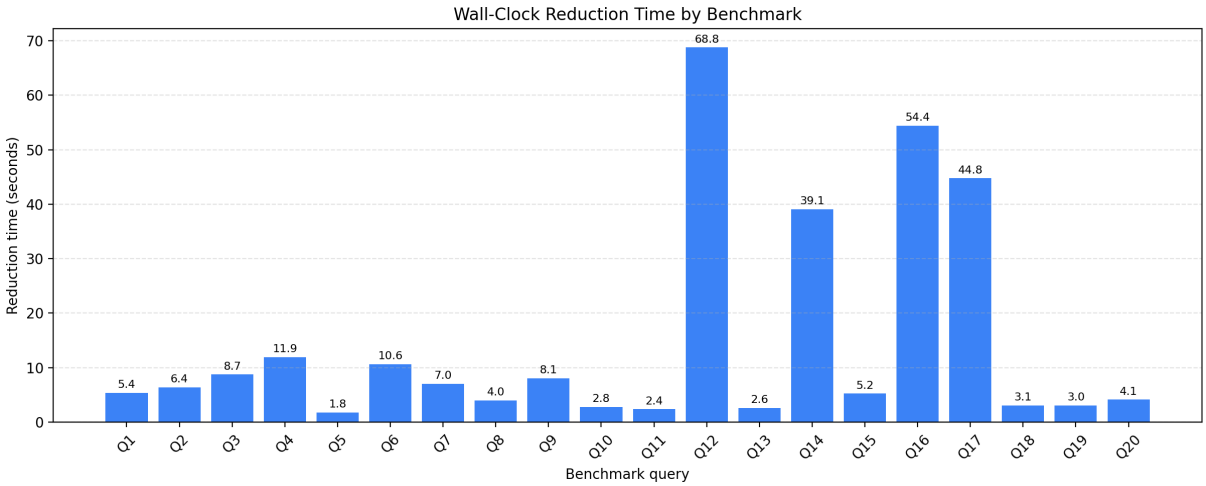

Figure 2: Speed of reduction across 20 benchmarks.

Note: The data is also presented in a tabular form in Appendix A.

For most queries, the reducer is reasonably fast, with the median reduction time across the 20 benchmarks being 5.9 seconds. However, the distribution is, once again, quite uneven. Some queries, especially those that were challenging to reduce, take much more time than most queries, and this raises the mean reduction time to 14.7 seconds. Among these queries are *Q12* and *Q14*, which were the most challenging queries to reduce (<0.2% token reduction), pointing to the reducer exploring many candidate queries that did not preserve oracle behaviour (i.e., the bug in the query). This "wasted effort" on certain queries is better presented in Figure 3, which clearly shows that the system performs the worst on *Q12* and *Q14*. Overall, the reduction time on the majority of queries is acceptable, but can be too long on certain queries, especially when resulting in low token reduction. This behaviour points towards some potential future improvements, including better candidate query prioritisation in the reduction strategy and the implementation of early stopping when AST mutations continuously violate oracle behaviour and cannot compute mutations that preserve the bug.

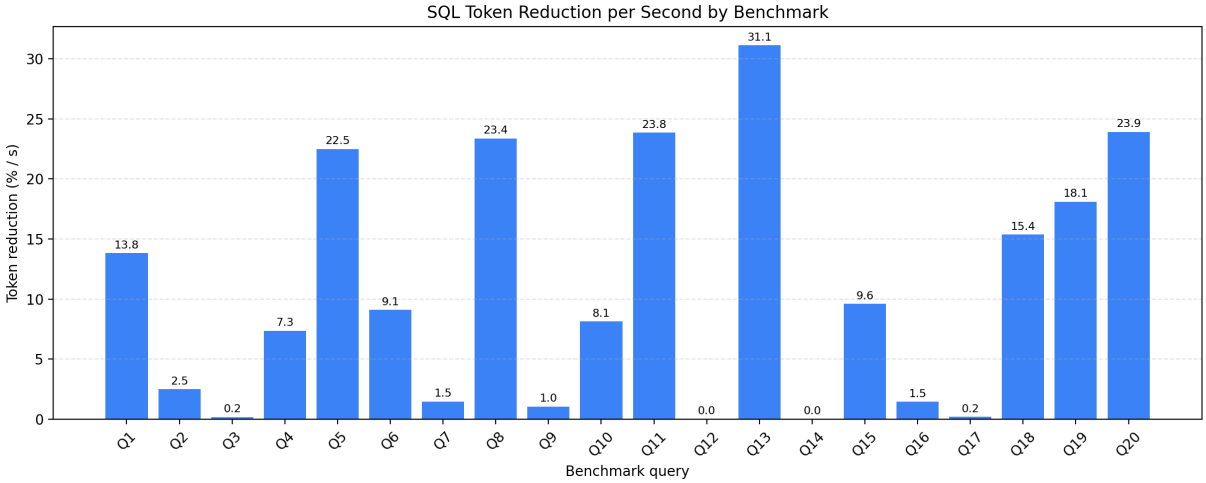

Figure 3: Token reduction percentage per second across 20 benchmarks.

4 Limitations

Although we used two different algorithms to cover our bases while implementing the overarching strategy, there are certainly some limitations with our approach and the implemented reducer.

As the *ddmin* algorithm does not assume the data follows any particular structure, applying it directly to token and statement streams means that a large portion of the intermediate reduction steps yield syntactically or semantically invalid queries. Transitioning to a fully syntax-aware framework, such as HDD [2], could significantly enhance

both reduction depth and execution speed by ensuring that most mutations respect the grammar of the language. Additionally, both strategies within the overarching algorithm are "incomplete" and do not guarantee global minimality as they explore a limited set of candidate reductions; *ddmin* due to early stopping and the AST-based approach due to a limited set of potential transformations.

With regards to performance, with the current implementation, each candidate query must be tested against the oracle, and for certain queries, the number of oracle calls can make up a substantial amount of the execution time. Both strategies within the overarching algorithm are affect by this; *ddmin* may test many invalid queries, whereas the AST-based strategy may test many syntactically-valid but semantically useless queries. However, as the current architectural phases execute sequentially, introducing concurrent oracle evaluations or parallel branch exploration could lead to large performance improvements.

5 Conclusion

In this project, we developed and evaluated an automated test-case reduction tool for SQL queries. We implemented two complementary reduction strategies, with the *ddmin* approach reducing queries by removing candidate chunks and the AST-based approach using SQL structure to generate syntax-aware candidate queries for reduction.

The evaluation of our system demonstrates that our tool successfully shrinks complex queries by up to 99% within a matter of seconds. These findings demonstrate the utility and efficiency of AST-based and delta-debugging reduction strategies when minimising complex test inputs generated by automated fuzzing frameworks. The runtime was acceptable for most queries, with a median wall-clock reduction time of 5.9 seconds per query.

On the other hand, we also observed some clear limitations with our approach. Our system was not able to reduce certain queries effectively, leading to a mean token reduction of only 46.27% per query. Additionally, our system required significantly more time to reduce a smaller number of benchmarks, with it sometimes not being able to reduce queries even after spending much time, leading to a mean reduction time of 14.7 seconds.

Overall, our reduction strategy seems to successfully reduce many queries in a reasonable amount of time, but faces challenges with particular types of queries. To improve upon this, future work could include a larger variety of AST transformations, better candidate prioritisation, and strategies that can transform large parts of the initial query quickly. These extensions would likely increase both reduction quality and execution speed across a wider range of queries.

6 Disclaimers

6.1 LLM Usage

We used LLMs to bounce ideas off as well as help with the scripting of the project. Additionally, we used to help with debugging tasks and refine the wording of this report.

6.2 Code Availability

The full source code for our reducer, including data used in this report, is available on GitHub at `https://github.com/lmeller-git/ast-testcase-reduce`.

References

[1] R. Hildebrandt and A. Zeller, "Simplifying failure-inducing input," presented at the ISSTA '00, ACM, Aug. 1, 2000, pp. 135–145, ISBN: 1-58113-266-2. DOI: 10.1145/347324.348938.

[2] G. Misherghi and Z. Su, "HDD: Hierarchical delta debugging," presented at the ICSE '06, ACM, May 28, 2006, pp. 142–151, ISBN: 978-1-59593-375-1. DOI: 10.1145/1134285.1134307.

Appendix: Supplementary Data

A Evaluation Metrics Data-points

Table 1: Token reduction (%) and wall-clock execution time (s) across 20 queries.

Query Number	Tokens Reduction (%)	Wall-clock Time (s)
1	74.54	5.39
2	16.01	6.41
3	1.47	8.75
4	87.54	11.91
5	39.71	1.77
6	96.77	10.64
7	10.34	7.04
8	93.33	3.99
9	8.43	8.06
10	22.78	2.80
11	56.14	2.35
12	0.00	68.78
13	80.14	2.58
14	0.20	39.19
15	50.00	5.21
16	79.01	54.40
17	8.28	44.77
18	46.89	3.05
19	54.98	3.04
20	98.78	4.13